

# Part A – System Design

## Architecture overview

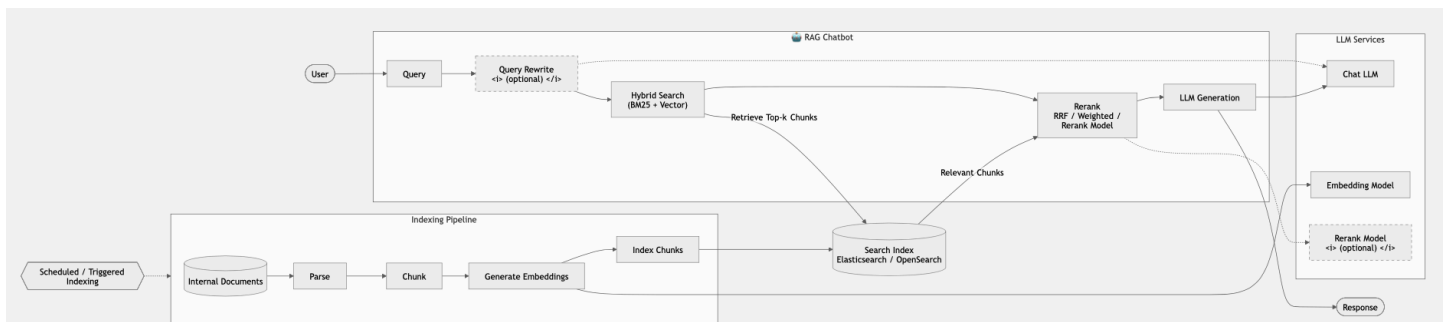
As users need to ask questions about internal documents with snappy responses, I assume a standard RAG chatbot is sufficient and there is no need for a complicated agentic system initially. Unless evaluation proves otherwise, adding agent complexity will increase latency and operational overhead.

The system mainly consists of an indexing pipeline and an online RAG pipeline. For indexing, documents are parsed, chunked, converted into embeddings, and stored in the search index. Since around 2,000 documents are updated monthly, scheduled indexing is sufficient. Real-time indexing can be added later if users require immediate updates.

For online queries, users send questions through the API service. Queries can optionally be rewritten before hybrid retrieval from the knowledge base. Retrieved chunks are reranked using RRF, weighted scoring, or an optional reranking model. The final context is passed to the existing LLM service to generate answers.

A search engine such as Elasticsearch/OpenSearch is suitable because it supports both keyword and vector search, enabling hybrid retrieval. A vector-only database is not preferred because exact keyword matching is still important for internal documents.

System design flowchart is as below, please see original image in the repo.



## Key decisions and tradeoffs

### Simple RAG instead of agentic workflow

Chosen because the use case is document question answering. Agents may improve complex reasoning but add latency and maintenance complexity.

### Hybrid search instead of vector-only search

Improves retrieval quality by combining semantic similarity and keyword matching.

Tradeoff: slightly more complexity in the retrieval pipeline.

### Batch indexing instead of real-time indexing

Chosen because document updates are monthly and the scale is relatively small.

Tradeoff: updated documents may not immediately appear in search results.

## Post-deployment monitoring

Before production, evaluate the chatbot against a test dataset with expected answers. Retrieval quality and generation quality should be evaluated separately:

- Poor retrieved chunks indicate retrieval issues.
- Relevant chunks but incorrect answers indicate prompt or model issues.

After deployment, monitor:

- Answer quality through sampled evaluation and user feedback.
- Retrieval relevance.
- End-to-end latency and error rates.
- Index freshness.

The system should support around 20 concurrent users through load testing and monitor whether shared GPU capacity becomes a bottleneck.

## Production failure mode

One failure mode that may only appear in production is poor retrieval quality caused by real user queries differing from the evaluation dataset.

Users may phrase questions differently from expected test cases, use abbreviations, or provide less context. Although the required information exists in the documents, the retrieval system may fail to find relevant chunks, resulting in inaccurate answers.

This can be detected through user feedback, sampled quality evaluation, and retrieval relevance monitoring. Mitigation includes improving query rewriting, tuning retrieval parameters, and expanding the evaluation dataset with real production queries.

## Part C – Opinion

Assuming I can reproduce the issue with user-provided queries, I will debug by identifying whether the problem comes from indexing, retrieval, or generation.

### **1. Check whether the LLM receives the correct context.**

I will inspect request logs containing the user query, retrieved chunks, metadata, and final prompt. I will verify that indexed chunks match the latest document versions by checking document timestamps/hashes. If outdated and new versions are both retrieved, I will fix indexing logic such as version handling and deduplication.

### **2. Check retrieval quality.**

If the correct document exists in the index but is not retrieved, I will inspect hybrid search scores and top-k results. Possible causes include ambiguous queries, poor chunking, or reranking filtering out relevant documents. Fixes include query rewriting, tuning chunk size/splitting strategy, and optimising reranking.

### **3. Check generation behaviour.**

If the retrieved context is correct, I will inspect the final prompt and LLM output for hallucination or instruction-following issues. I will check prompt changes, model version, and context length limits before considering model replacement.